COMPILER DESIGN | ASSIGNMENT 35

UB TIME BOMB DETECTOR

Fixed & Corrected Build Plan for Windows

This document supersedes the original build plan.
All identified bugs have been fixed with corrected code.
Each fix is clearly marked with **BUG** (original) and **FIX** (corrected) blocks.

#	Bug	Severity	Affected Phase
1	Shared LLVMContext for both modules	CRITICAL	Phase 1 / main.cpp
2	hasLoadBeforeStore() uses users() not BB order	HIGH	Phase 3
3	hasSignedWrapAdded() produces false positives	HIGH	Phase 3
4	touch command not available on Windows	MEDIUM	Phase 0
5	Inlined functions silently skipped with no log	MEDIUM	Phase 2
6	Missing llvm::demangle() in reporter	LOW	Phase 4

! All Bugs & Fixes — Detailed Explanation

Read this section before starting any phase

This section explains every bug found in the original plan, why it matters, and the exact corrected code. Reference these fixes as you work through each phase.

Bug 1 — Shared LLVMContext (CRITICAL)

Why it matters: An **llvm::LLVMContext** owns all LLVM type and metadata objects. When you compile two separate modules (one at -O0, one at -O2) into the *same* context, their type tables collide. The second compilation overwrites metadata from the first, producing silent data corruption — you get wrong block counts, wrong type pointers, and iterator crashes that are nearly impossible to debug.

BUG: Original main.cpp — one shared context for both modules

```
llvm::LLVMContext ctx;
auto M0 = compileAt(file, 0, ctx);  // populates ctx with O0 types
auto M2 = compileAt(file, 2, ctx);  // CONFLICT: O2 types collide with O0
// result: corrupt metadata, wrong block counts, random crashes
```

FIX: Use separate contexts — one per compilation

```
llvm::LLVMContext ctx0, ctx2;           // SEPARATE contexts
auto M0 = compileAt(file, 0, ctx0);    // O0 module in ctx0
auto M2 = compileAt(file, 2, ctx2);    // O2 module in ctx2, no conflict
// Now match functions by NAME (string), not by pointer equality
```

■■ Important consequence of this fix

With separate contexts, you CANNOT compare llvm::Type* pointers across modules. Always match functions by getName() string. The detectChanges() function already does this correctly. Do not add any cross-module

Bug 2 — hasLoadBeforeStore() Uses Wrong Iteration Order (HIGH)

Why it matters: llvm::Value::users() returns users in *definition order*, not *execution order*. In LLVM IR, a store that appears later in the source code can appear earlier in the user list. This means the original code would miss real uninitialized reads — it would see a store first in the user list even though the load executes first at runtime.

BUG: Original — iterates users() which is NOT program order

```
for (auto* U : AI->users()) {              // users() = unordered!
    if (llvm::isa<llvm::StoreInst>(U))  { seenStore = true; break; }
    if (llvm::isa<llvm::LoadInst>(U) && !seenStore) {
        return true;  // WRONG: may see store before load even if load runs first
    }
}
```

FIX: Walk basic block instructions IN ORDER (program order)

```
static bool hasLoadBeforeStore(llvm::Function* F2,
                               llvm::Instruction** culpritOut) {
    for (auto& BB : *F2) {
        for (auto& I : BB) {
```

Bug 3 — hasSignedWrapAdded() False Positives (HIGH)

Why it matters: The original code flags ANY function that contains an nsw flag at -O2, even if no behavioral change occurred. The LLVM optimizer adds nsw to arithmetic instructions whenever it can *prove* no overflow happens — including in completely safe code. Without correlating the nsw flag with an actual branch elimination, you will report false positives on almost every non-trivial function.

BUG: Original — any nsw flag triggers a report

```
static bool hasSignedWrapAdded(llvm::Function* F_O0, llvm::Function* F_O2) {
    for (auto& BB : *F_O2)
        for (auto& I : BB)
            if (auto* B = llvm::dyn_cast<llvm::BinaryOperator>(&I))
                if (B->hasNoSignedWrap()) return true;  // FALSE POSITIVE!
    return false;
}
// Problem: 'int safe(int a) { return a * 2; }' also gets nsw at O2
// but no branch is eliminated -- it is NOT a time bomb
```

FIX: Require BOTH nsw flag AND branch elimination (true positive)

```
static int countConditionalBranches(llvm::Function& F) {
    int n = 0;
    for (auto& BB : F)
        for (auto& I : BB)
            if (auto* BI = llvm::dyn_cast<llvm::BranchInst>(&I))
                if (BI->isConditional()) n++;
    return n;
}

static bool hasSignedWrapAdded(llvm::Function* F_O0, llvm::Function* F_O2) {
    bool nswFound = false;
    for (auto& BB : *F_O2)
        for (auto& I : BB)
            if (auto* B = llvm::dyn_cast<llvm::BinaryOperator>(&I))
                if (B->hasNoSignedWrap()) { nswFound = true; break; }
    // Only a time bomb if nsw appeared AND branches were removed
    int br0 = countConditionalBranches(*F_O0);
    int br2 = countConditionalBranches(*F_O2);
    return nswFound && (br2 < br0);  // BOTH conditions required
}
```

Bug 4 — 'touch' Command Not on Windows (MEDIUM)

Why it matters: The original Phase 0.2 uses the Unix touch command to create empty files. This command does not exist in Windows PowerShell or Command Prompt by default. Running the script verbatim will produce 'command not found' errors before you write a single line of C++.

BUG: Original Phase 0.2 — uses touch (Unix only)

```
touch CMakeLists.txt README.md
touch include/{DiffEngine.h,BehaviorDiff.h,UBClassifier.h,Reporter.h}
# ERROR on Windows: 'touch' is not recognized
```

FIX: Windows PowerShell equivalent using New-Item

```
# Create all files using native PowerShell
@('CMakeLists.txt','README.md') | ForEach-Object {
    New-Item $_ -ItemType File -Force | Out-Null
}
@('DiffEngine.h','BehaviorDiff.h','UBClassifier.h','Reporter.h') |
    ForEach-Object { New-Item "include/$_" -ItemType File -Force | Out-Null }
@('main.cpp','DiffEngine.cpp','BehaviorDiff.cpp',
 'UBClassifier.cpp','Reporter.cpp') |
    ForEach-Object { New-Item "src/$_" -ItemType File -Force | Out-Null }
@('signed_overflow.c','null_deref.c','aliasing.c','uninit.c') |
    ForEach-Object { New-Item "tests/$_" -ItemType File -Force | Out-Null }
```

Bug 5 — Inlined Functions Silently Skipped (MEDIUM)

Why it matters: At -O2, the compiler inlines small functions — they disappear from the module entirely. The original detectChanges() silently skips functions missing from M_O2 with a continue. This means time bombs in inlined functions are completely invisible to the tool with no indication to the developer that they were skipped.

BUG: Original — silently skips inlined functions

```
auto* F2 = M_O2->getFunction(F0.getName());
if (!F2 || F2->isDeclaration()) continue;  // silent skip -- no log!
// Developer never knows this function was inlined away
```

FIX: Log inlined functions explicitly

```
auto* F2 = M_O2->getFunction(F0.getName());
if (!F2 || F2->isDeclaration()) {
    // Explicitly log so developer knows this was skipped
    llvm::outs() << "[INFO] Function '" << F0.getName()
                 << "' was inlined at -O2; skipping CFG diff.\n";
    continue;
}
```

Bug 6 — Missing llvm::demangle() in Reporter (LOW)

Why it matters: Without demangling, C++ function names appear as mangled symbols like _Z5f_ptrPi in the report instead of the readable f_ptr(int*). For C files it is less critical but for C++ source files the output becomes unreadable.

BUG: Original Reporter.cpp — prints raw mangled name

```
llvm::errs() << "[UB TIME BOMB #" << n++ << "] "
             << locationStr(r.culprit) << "\n";
// Prints: [UB TIME BOMB #1] signed_overflow.c:3
// But function shown elsewhere as: _Z1fi  <-- unreadable
```

FIX: Add demangle() call for readable function names

```
#include <llvm/Demangle/Demangle.h>

// In emitReport(), change the header line to:
std::string readable = llvm::demangle(r.diff.name);
```

0 Phase 0 — Environment Setup & IR Literacy

Run every command manually before writing C++ code

Before writing any C++ code you must have LLVM installed and be comfortable reading IR files by hand. This phase is entirely manual — no code to write.

0.1 Install Dependencies (Windows)

POWERSHELL

```
# Option A: winget (recommended)
winget install LLVM.LLVM
winget install Kitware.CMake
winget install Ninjabuild.Ninja

# Option B: MSYS2 (if winget is unavailable)
pacman -S mingw-w64-x86_64-llvm mingw-w64-x86_64-clang
pacman -S mingw-w64-x86_64-cmake mingw-w64-x86_64-ninja

# Verify versions (must be 14+)
llvm-config --version
clang --version
cmake --version
```

0.2 Create Project Skeleton [FIX #4 applied — no 'touch']

■ Bug 4 Fixed Here

All file creation now uses New-Item instead of touch, which does not exist on Windows.

POWERSHELL

```
# Create directory tree
$dirs = @(
  'ub-time-bomb/include',
  'ub-time-bomb/src',
  'ub-time-bomb/tests/expected_outputs',
  'ub-time-bomb/eval/cve_cases'
)
$dirs | ForEach-Object { New-Item -ItemType Directory -Force -Path $_ }
Set-Location ub-time-bomb

# Create root files
@('CMakeLists.txt','README.md') |
    ForEach-Object { New-Item $_ -ItemType File -Force | Out-Null }

# Create header files
@('DiffEngine.h','BehaviorDiff.h','UBClassifier.h','Reporter.h') |
    ForEach-Object { New-Item "include/$_" -ItemType File -Force | Out-Null }

# Create source files
@('main.cpp','DiffEngine.cpp','BehaviorDiff.cpp',
  'UBClassifier.cpp','Reporter.cpp') |
    ForEach-Object { New-Item "src/$_" -ItemType File -Force | Out-Null }

# Create test files
@('signed_overflow.c','null_deref.c','aliasing.c','uninit.c') |
    ForEach-Object { New-Item "tests/$_" -ItemType File -Force | Out-Null }
```

0.3 Write the Four Canonical Test Inputs

```c
// tests/signed_overflow.c
// Expected: -O2 eliminates the branch; always returns 1
int f(int x) {
    return x + 1 > x;
}
```

```c
// tests/null_deref.c
// Expected: -O2 removes the null-check block
int use_ptr(int *p) {
    int val = *p;   // dereference before null check -- UB if p==NULL
    if (p == 0) return -1;
    return val;
}
```

```c
// tests/aliasing.c
// Expected: -O2 may reorder load/store
int alias_bug(int *ip) {
    float *fp = (float *)ip;
    *fp = 1.0f;
    return *ip;
}
```

```c
// tests/uninit.c
// Expected: -O2 treats x as undefined; may omit the add
int f(void) {
    int x;
    return x + 1;
}
```

0.4 Manually Emit and Diff the IR

```powershell
cd tests

# Emit IR at both levels for signed_overflow
clang -O0 -g -S -emit-llvm signed_overflow.c -o signed_overflow_O0.ll
clang -O2 -g -S -emit-llvm signed_overflow.c -o signed_overflow_O2.ll
fc signed_overflow_O0.ll signed_overflow_O2.ll

# Look for: 'add nsw' at O2, branch count dropping to 0

# Emit IR for null_deref
clang -O0 -g -S -emit-llvm null_deref.c -o null_deref_O0.ll
clang -O2 -g -S -emit-llvm null_deref.c -o null_deref_O2.ll
fc null_deref_O0.ll null_deref_O2.ll

# Look for: basic block count dropping from 3 to 1

# Record your observations -- these become your test oracles:
# signed_overflow : O0 blocks = ?   O2 blocks = ?
# null_deref      : O0 blocks = ?   O2 blocks = ?
```

PHASE GATE CHECKLIST

- [ ] LLVM 14+ installed; llvm-config --version prints without error
- [ ] Four .c test files created in tests/ using New-Item (not touch)
- [ ] All four .ll files emitted at both -O0 and -O2 with no compiler errors
- [ ] You have manually noted basic-block counts for each test case
- [ ] You can see 'add nsw' in signed_overflow_O2.ll and NOT in _O0.ll

1 Phase 1 — Differential Compilation Engine

Deliverable 1 — Programmatic IR capture with SEPARATE contexts

■ Critical Fix Applied in This Phase (Bug 1)

main.cpp uses TWO separate LLVMContext objects — ctx0 for O0 and ctx2 for O2. This is the most important change from the original plan. Do not merge them.

1.1 CMakeLists.txt (unchanged from original)

CMAKE

```
cmake_minimum_required(VERSION 3.14)
project(UBTimeBombDetector)

find_package(LLVM REQUIRED CONFIG)
find_package(Clang REQUIRED CONFIG)

include_directories(${LLVM_INCLUDE_DIRS} ${CLANG_INCLUDE_DIRS})
add_definitions(${LLVM_DEFINITIONS})

add_executable(ub-detector
    src/main.cpp  src/DiffEngine.cpp  src/BehaviorDiff.cpp
    src/UBClassifier.cpp  src/Reporter.cpp
)

llvm_map_components_to_libnames(llvm_libs
    support core irreader passes analysis)

target_link_libraries(ub-detector
    ${llvm_libs}
    clangFrontend clangCodeGen clangDriver
    clangSerialization clangParse clangSema
    clangAnalysis clangAST clangBasic
)
set_target_properties(ub-detector PROPERTIES
    CXX_STANDARD 17 CXX_STANDARD_REQUIRED YES)
```

1.2 include/DiffEngine.h (unchanged from original)

C++

```
#pragma once
#include <llvm/IR/Module.h>
#include <memory>
#include <string>

// Compiles 'filepath' at the given optimization level (0 or 2)
// and returns the resulting llvm::Module.
// ctx must be a SEPARATE context for each call -- see main.cpp
std::unique_ptr<llvm::Module>
compileAt(const std::string& filepath, int optLevel,
          llvm::LLVMContext& ctx);
```

1.3 src/DiffEngine.cpp (unchanged from original)

```
C++

#include "DiffEngine.h"
#include <clang/CodeGen/CodeGenAction.h>
#include <clang/Frontend/CompilerInstance.h>
#include <clang/Frontend/CompilerInvocation.h>
#include <clang/Basic/DiagnosticOptions.h>
#include <clang/Basic/TargetInfo.h>
#include <llvm/Support/TargetSelect.h>
#include <llvm/IR/LLVMContext.h>
#include <vector>
#include <string>

std::unique_ptr<llvm::Module>
compileAt(const std::string& filepath, int optLevel,
          llvm::LLVMContext& ctx)
{
    llvm::InitializeAllTargets();
    llvm::InitializeAllTargetMCs();
    llvm::InitializeAllAsmPrinters();
    llvm::InitializeAllAsmParsers();

    clang::CompilerInstance CI;
    CI.createDiagnostics();
    if (!CI.hasDiagnostics()) return nullptr;

    std::vector<const char*> args = {
        "clang",
        filepath.c_str(),
        ("-O" + std::to_string(optLevel)).c_str(),
        "-g", "-S", "-emit-llvm",
        "-fno-inline",
        "-Wno-everything"
    };

    auto invocation = std::make_shared<clang::CompilerInvocation>();
    clang::CompilerInvocation::CreateFromArgs(
        *invocation,
        llvm::ArrayRef<const char*>(args.data(), args.size()),
        CI.getDiagnostics());
    CI.setInvocation(invocation);

    clang::EmitLLVMOnlyAction action(&ctx);
    if (!CI.ExecuteAction(action)) return nullptr;
    return action.takeModule();
}
```

1.4 src/main.cpp — Phase 1 Harness [FIX #1 applied]

> **■ Bug 1 Fixed Here**
>
> ctx0 and ctx2 are declared separately. Each compileAt() call gets its own context. Function matching in all subsequent phases uses getName() strings.

```
C++

#include "DiffEngine.h"
#include <llvm/IR/LLVMContext.h>
#include <llvm/Support/raw_ostream.h>

int main(int argc, char* argv[]) {
```

1.5 Build & Test

POWERSHELL

```
mkdir build; cd build
cmake .. -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build . --config Debug

.\ub-detector.exe ..\tests\signed_overflow.c
# Expected:
# [Phase 1] -O0 module: ...
#   Functions: 1
# [Phase 1] -O2 module: ...
#   Functions: 1

.\ub-detector.exe ..\tests\null_deref.c
.\ub-detector.exe ..\tests\aliasing.c
.\ub-detector.exe ..\tests\uninit.c
# All four must run without crashing
```

PHASE GATE CHECKLIST

- ☐ cmake --build produces ub-detector.exe with zero errors
- ☐ .\ub-detector.exe ..\tests\signed_overflow.c prints two module names
- ☐ Runs cleanly on all four .c test files without crashing
- ☐ Function counts printed for -O0 and -O2 are equal (same source, diff IR)

2 Phase 2 — Behavioral Change Detector

Deliverable 2 — CFG diff with inlined-function logging

■ Bug 5 Fixed Here

Functions missing from M_O2 due to inlining are now logged with [INFO] instead of silently skipped. Developer always knows what was skipped.

2.1 include/BehaviorDiff.h (unchanged from original)

C++

```
#pragma once
#include <llvm/IR/Module.h>
#include <string>
#include <vector>

struct FunctionDiff {
    std::string name;
    int blocks_O0, blocks_O2;
    int branches_O0, branches_O2;
    bool blockLoss()  const { return blocks_O2 < blocks_O0; }
    bool branchLoss() const { return branches_O2 < branches_O0; }
    bool hasDiff()    const { return blockLoss() || branchLoss(); }
};

std::vector<FunctionDiff>
detectChanges(llvm::Module* M_O0, llvm::Module* M_O2);
```

2.2 src/BehaviorDiff.cpp [FIX #5 applied — logs inlined functions]

C++

```
#include "BehaviorDiff.h"
#include <llvm/IR/Instructions.h>
#include <llvm/IR/Function.h>
#include <llvm/Support/raw_ostream.h>

static int countConditionalBranches(llvm::Function& F) {
    int count = 0;
    for (auto& BB : F)
        for (auto& I : BB)
            if (auto* BI = llvm::dyn_cast<llvm::BranchInst>(&I))
                if (BI->isConditional()) ++count;
    return count;
}

std::vector<FunctionDiff>
detectChanges(llvm::Module* M_O0, llvm::Module* M_O2)
{
    std::vector<FunctionDiff> results;
    for (auto& F0 : *M_O0) {
        if (F0.isDeclaration()) continue;

        auto* F2 = M_O2->getFunction(F0.getName());

        // FIX #5: Log inlined functions instead of silent skip
        if (!F2 || F2->isDeclaration()) {
            llvm::outs() << "[INFO] " << F0.getName()
```

2.3 Update src/main.cpp — integrate Phase 2

```C++
// Add includes at top of main.cpp:
#include "BehaviorDiff.h"

// After the Phase 1 smoke output, add:
auto diffs = detectChanges(M0.get(), M2.get());

if (diffs.empty()) {
    llvm::outs() << "No behavioral changes detected.\n";
    return 0;
}

for (auto& d : diffs) {
    llvm::outs() << "\n[CHANGED FUNCTION] " << d.name << "\n";
    llvm::outs() << "  Basic blocks : O0=" << d.blocks_O0
                 << " O2=" << d.blocks_O2
                 << (d.blockLoss()  ? " <-- BLOCK LOSS"   : "") << "\n";
    llvm::outs() << "  Cond branches: O0=" << d.branches_O0
                 << " O2=" << d.branches_O2
                 << (d.branchLoss() ? " <-- BRANCH ELIM" : "") << "\n";
}
```

2.4 Expected Phase 2 Outputs

Test File	Expected Output
tests/signed_overflow.c	Function f: blocks drop; branches_O0=1, branches_O2=0 --> BRANCH ELIM
tests/null_deref.c	Function use_ptr: blocks_O0=3, blocks_O2=1 --> BLOCK LOSS + BRANCH ELIM
tests/aliasing.c	May NOT flag -- aliasing is instruction reorder, not CFG change. Acceptable.
tests/uninit.c	Blocks may collapse. At minimum: no crash.

PHASE GATE CHECKLIST

- ☐ detectChanges() returns non-empty for signed_overflow.c and null_deref.c
- ☐ Block counts and branch counts printed correctly for all four test files
- ☐ Inlined functions print [INFO] message instead of silently disappearing
- ☐ No false positives on 'int add(int a, int b){ return a+b; }'
- ☐ aliasing.c and uninit.c run without crash even if no diff flagged

3 Phase 3 — UB Pattern Classifier

Deliverable 3 — Two bugs fixed in this phase

■ Two Fixes Applied in This Phase (Bugs 2 & 3)

hasLoadBeforeStore() now walks basic block instructions in order (not users()). hasSignedWrapAdded() now requires BOTH nsw flag AND branch elimination.

3.1 UB Categories and IR Signatures (unchanged)

UB Category	IR Signature
Signed Integer Overflow	At -O2: arithmetic gains 'nsw'/'nuw' flag absent at -O0 AND a branch was eliminated
Null Pointer Dereference	At -O2: a basic block with icmp eq/ne ptr, null disappears; block count drops
Strict Aliasing	Load of type T reordered past store through type T* (instruction reorder, not CFG)
Uninitialized Variable	alloca with load before any store IN EXECUTION ORDER (not user-list order)

3.2 include/UBClassifier.h (unchanged from original)

C++

```
#pragma once
#include "BehaviorDiff.h"
#include <llvm/IR/Module.h>
#include <string>
#include <vector>

enum class UBKind {
    None, SignedOverflow, NullDeref,
    StrictAliasing, UninitRead, Unknown,
};

struct UBResult {
    FunctionDiff diff;
    UBKind kind;
    std::string kindStr() const;
    std::string detail;
    llvm::Instruction* culprit = nullptr;
};

std::vector<UBResult>
classifyDiffs(const std::vector<FunctionDiff>& diffs,
              llvm::Module* M_O0, llvm::Module* M_O2);
```

3.3 src/UBClassifier.cpp [FIX #2 + FIX #3 applied]

C++

```
#include "UBClassifier.h"
#include <llvm/IR/Instructions.h>
#include <llvm/IR/InstrTypes.h>
#include <llvm/IR/Constants.h>

std::string UBResult::kindStr() const {
    switch (kind) {
```

C++

```
// FIX #3: Require nsw flag AND branch elimination (not just nsw alone)
static int countConditionalBranches(llvm::Function& F) {
    int n = 0;
    for (auto& BB : F)
        for (auto& I : BB)
            if (auto* BI = llvm::dyn_cast<llvm::BranchInst>(&I))
                if (BI->isConditional()) n++;
    return n;
}

static bool hasSignedWrapAdded(llvm::Function* F_O0, llvm::Function* F_O2) {
    bool nswFound = false;
    for (auto& BB : *F_O2)
        for (auto& I : BB)
            if (auto* B = llvm::dyn_cast<llvm::BinaryOperator>(&I))
                if (B->hasNoSignedWrap()) { nswFound = true; break; }
    // BOTH conditions must be true -- prevents false positives
    int br0 = countConditionalBranches(*F_O0);
    int br2 = countConditionalBranches(*F_O2);
    return nswFound && (br2 < br0);
}
```

C++

```
static bool hasNullCheckRemoved(llvm::Function* F_O0, llvm::Function* F_O2,
                                llvm::Instruction** culpritOut) {
    if ((int)F_O2->size() >= (int)F_O0->size()) return false;
    for (auto& BB : *F_O0)
        for (auto& I : BB)
            if (auto* CI = llvm::dyn_cast<llvm::ICmpInst>(&I)) {
                if (CI->getPredicate() == llvm::ICmpInst::ICMP_EQ ||
                    CI->getPredicate() == llvm::ICmpInst::ICMP_NE)
                    for (auto* op : CI->operand_values())
                        if (llvm::isa<llvm::ConstantPointerNull>(op)) {
                            if (culpritOut) *culpritOut = &I;
                            return true;
                        }
            }
    return false;
}
```

C++

```
// FIX #2: Walk BB in instruction order, NOT users() which is unordered
static bool hasLoadBeforeStore(llvm::Function* F2,
                               llvm::Instruction** culpritOut) {
    for (auto& BB : *F2) {
        for (auto& I : BB) {
            if (auto* AI = llvm::dyn_cast<llvm::AllocaInst>(&I)) {
                bool seenStore = false;
                // Iterate BB instructions sequentially (program order)
                for (auto& J : *AI->getParent()) {
                    if (auto* SI = llvm::dyn_cast<llvm::StoreInst>(&J))
                        if (SI->getPointerOperand() == AI)
                            { seenStore = true; break; }
                    if (auto* LI = llvm::dyn_cast<llvm::LoadInst>(&J))
                        if (LI->getPointerOperand() == AI && !seenStore) {
                            if (culpritOut) *culpritOut = LI;
                            return true;
                        }
```

C++

```
std::vector<UBResult>
classifyDiffs(const std::vector<FunctionDiff>& diffs,
              llvm::Module* M_O0, llvm::Module* M_O2)
{
    std::vector<UBResult> results;
    for (auto& d : diffs) {
        auto* F0 = M_O0->getFunction(d.name);
        auto* F2 = M_O2->getFunction(d.name);
        if (!F0 || !F2) continue;

        UBResult r;
        r.diff = d;
        llvm::Instruction* culprit = nullptr;

        // Priority: overflow > null > uninit > unknown
        if (hasSignedWrapAdded(F0, F2)) {
            r.kind   = UBKind::SignedOverflow;
            r.detail = "'add nsw' at -O2 + branch eliminated: "
                       "overflow assumption exploited.";
            for (auto& BB : *F2)
                for (auto& I : BB)
                    if (auto* B = llvm::dyn_cast<llvm::BinaryOperator>(&I))
                        if (B->hasNoSignedWrap()) { culprit = &I; break; }
        } else if (hasNullCheckRemoved(F0, F2, &culprit)) {
            r.kind   = UBKind::NullDeref;
            r.detail = "Null check at -O0 removed at -O2: pointer "
                       "assumed non-null due to prior dereference.";
        } else if (hasLoadBeforeStore(F2, &culprit)) {
            r.kind   = UBKind::UninitRead;
            r.detail = "alloca loaded before store in execution order: "
                       "value is indeterminate (poison).";
        } else {
            r.kind   = UBKind::Unknown;
            r.detail = "CFG changed but pattern unmatched. Possible "
                       "aliasing UB -- inspect IR manually.";
        }
        r.culprit = culprit;
        results.push_back(r);
    }
    return results;
}
```

PHASE GATE CHECKLIST

- [ ] signed_overflow.c classified as SignedOverflow (nsw + branch elim both present)
- [ ] null_deref.c classified as NullDeref
- [ ] uninit.c classified as UninitRead (or Unknown -- not SignedOverflow)
- [ ] safe 'int f(int a){return a*2;}' produces NO output (false positive test)
- [ ] No crashes on any of the four canonical test files

4 Phase 4 — Source-Level Reporter

Deliverable 4 — With demangled function names

■ Bug 6 Fixed Here

llvm::demangle() is now called on function names before printing. C++ mangled names like _Z1fi now appear as f(int) in the report.

4.1 include/Reporter.h (unchanged)

C++

```
#pragma once
#include "UBClassifier.h"
#include <string>
#include <vector>

void emitReport(const std::string& sourceFile,
                const std::vector<UBResult>& results);
```

4.2 src/Reporter.cpp [FIX #6 applied — demangle added]

C++

```
#include "Reporter.h"
#include <llvm/IR/DebugInfoMetadata.h>
#include <llvm/Support/raw_ostream.h>
#include <llvm/Demangle/Demangle.h>   // FIX #6: for readable names

static std::string locationStr(llvm::Instruction* I) {
    if (!I) return "unknown location";
    if (llvm::DILocation* Loc = I->getDebugLoc())
        return Loc->getFilename().str() + ":" +
               std::to_string(Loc->getLine());
    if (auto* BB = I->getParent())
        for (auto it = BB->begin(); it != BB->end(); ++it)
            if (auto* Loc = it->getDebugLoc().get())
                return Loc->getFilename().str() + ":" +
                       std::to_string(Loc->getLine());
    return "no debug info (compile with -g)";
}

static std::string fix(UBKind k) {
    switch (k) {
        case UBKind::SignedOverflow:
            return "Use unsigned arithmetic or __builtin_add_overflow().";
        case UBKind::NullDeref:
            return "Move null check BEFORE first pointer dereference.";
        case UBKind::UninitRead:
            return "Initialize the variable at declaration.";
        case UBKind::StrictAliasing:
            return "Use memcpy() for type-punning or -fno-strict-aliasing.";
        default:
            return "Inspect IR: clang -O0/-O2 -g -S -emit-llvm <file>";
    }
}

void emitReport(const std::string& sourceFile,
                const std::vector<UBResult>& results)
{
    llvm::errs() << "\n=== UB Time Bomb Detector Report ===\n";
    llvm::errs() << "Analyzed: " << sourceFile << "\n\n";
    if (results.empty()) {
        llvm::errs() << "No UB time bombs found.\n";
        return;
    }
    int n = 1;
    for (auto& r : results) {
        // FIX #6: demangle for readable C++ names
        std::string readable = llvm::demangle(r.diff.name);
        llvm::errs() << "[UB TIME BOMB #" << n++ << "] "
                     << readable << " @ "
                     << locationStr(r.culprit) << "\n";
        llvm::errs() << "  UB Category  : " << r.kindStr() << "\n";
        llvm::errs() << "  Detail       : " << r.detail  << "\n";
        llvm::errs() << "  -O0 blocks   : " << r.diff.blocks_O0
                     << "  -O2 blocks: " << r.diff.blocks_O2 << "\n";
        llvm::errs() << "  -O0 branches : " << r.diff.branches_O0
                     << "  -O2 branches: " << r.diff.branches_O2 << "\n";
        llvm::errs() << "  Risk         : HIGH\n";
        llvm::errs() << "  Fix          : " << fix(r.kind) << "\n\n";
    }
    llvm::errs() << "Total time bombs: " << results.size() << "\n";
}
```

4.3 Final main.cpp — all four phases wired

C++

```
#include "DiffEngine.h"
#include "BehaviorDiff.h"
#include "UBClassifier.h"
#include "Reporter.h"
#include <llvm/IR/LLVMContext.h>
#include <llvm/Support/raw_ostream.h>

int main(int argc, char* argv[]) {
    if (argc < 2) {
        llvm::errs() << "Usage: ub-detector <source.c>\n";
        return 1;
    }
    std::string file = argv[1];

    // FIX #1: Always use separate contexts
    llvm::LLVMContext ctx0, ctx2;
    auto M0 = compileAt(file, 0, ctx0);
    auto M2 = compileAt(file, 2, ctx2);

    if (!M0 || !M2) {
        llvm::errs() << "Compilation failed for: " << file << "\n";
        return 1;
    }

    auto diffs   = detectChanges(M0.get(), M2.get());
    auto results = classifyDiffs(diffs, M0.get(), M2.get());
    emitReport(file, results);

    return results.empty() ? 0 : 2;  // exit 2 = bombs found
}
```

4.4 Build & Test

POWERSHELL

```
cd build; cmake --build . --config Debug

.\ub-detector.exe ..\tests\signed_overflow.c 2>&1
# Expected:
# [UB TIME BOMB #1] f(int) @ signed_overflow.c:3
#   UB Category  : Signed Integer Overflow
#   Fix          : Use unsigned arithmetic or __builtin_add_overflow()

.\ub-detector.exe ..\tests\null_deref.c 2>&1
# Expected:
# [UB TIME BOMB #1] use_ptr(int*) @ null_deref.c:<line>
#   UB Category  : Null Pointer Dereference

# Test exit codes
.\ub-detector.exe ..\tests\signed_overflow.c
echo "exit: $LASTEXITCODE"   # must be 2

'int f(int a){return a;}' | Set-Content "$env:TEMP\clean.c"
.\ub-detector.exe "$env:TEMP\clean.c"
echo "exit: $LASTEXITCODE"   # must be 0
```

PHASE GATE CHECKLIST

- [ ] Full report with file:line for signed_overflow.c (line must match source)
- [ ] Full report with file:line for null_deref.c
- [ ] Function names appear demangled (e.g. f(int) not _Z1fi)
- [ ] Exit code 2 when bombs found; exit code 0 when none
- [ ] Fix suggestion printed for every detected UB type
- [ ] No crash on any of the four canonical test files

5 Phase 5 — Real-World CVE Evaluation

Deliverable 5 — unchanged, honest evaluation required

No bug fixes are required in this phase. The CVE test cases and evaluation criteria are identical to the original plan. Run all five cases, document every outcome honestly — including cases your tool misses.

5.1 CVE Test Case Files

```c
// eval/cve_cases/gcc_bug_30475.c -- Signed Overflow, Linux Kernel
#include <limits.h>
int check_range(int start, int end) {
    for (int i = start; i < end; i++) {
        if (i + 1 < 0) return -1;  // overflow check removed at O2
    }
    return 0;
}
int main(void) { return check_range(INT_MAX - 2, INT_MAX); }
```

```c
// eval/cve_cases/cve_2009_1897.c -- Null Pointer Dereference
typedef struct { int val; } Obj;
int process(Obj *obj) {
    int v = obj->val;  // deref before null check
    if (!obj) return -1;  // removed at O2
    return v;
}
int main(void) { Obj o = {42}; return process(&o); }
```

```c
// eval/cve_cases/openssl_alias.c -- Strict Aliasing
int alias_cmp(int *ip) {
    float *fp = (float *)ip;
    *fp = 3.14f;
    return *ip;  // may see stale value at O2
}
int main(void) { int x = 0; return alias_cmp(&x); }
```

```c
// eval/cve_cases/dead_code.c -- Dead Code Elimination
#include <stdlib.h>
void* safe_alloc(size_t n) {
    void *p = malloc(n);
    if ((char*)p + n < (char*)p) {  // pointer overflow check -- UB
        free(p); return NULL;
    }
    return p;
}
int main(void) { free(safe_alloc(64)); return 0; }
```

```c
// eval/cve_cases/loop_wrap.c -- Signed Overflow in Loop
int count_up(int limit) {
    int count = 0;
```

5.2 Run the Evaluation

POWERSHELL

```
cd build
Get-ChildItem ..\eval\cve_cases\*.c | ForEach-Object {
    Write-Host "==== $_ ===="
    .\ub-detector.exe $_.FullName 2>&1
    Write-Host ""
}
```

5.3 Expected Results

CVE File	Expected Outcome	Reason
gcc_bug_30475.c	CAUGHT -- SignedOverflow	i+1 gets nsw flag; loop-body branch eliminated
cve_2009_1897.c	CAUGHT -- NullDeref	null check block removed at -O2
openssl_alias.c	PARTIAL / UNKNOWN	Aliasing = instruction reorder, not CFG change. Document as future work (MemorySSA).
dead_code.c	CAUGHT or UNKNOWN	Pointer arithmetic overflow may trigger block loss. Document outcome.
loop_wrap.c	CAUGHT -- SignedOverflow	Same nsw pattern as gcc_bug_30475

■ Document EVERY case

The assignment requires honest evaluation. Noting that strict aliasing requires MemorySSA (beyond scope) is academically correct. Never claim 100% detection.

PHASE GATE CHECKLIST

- ☐ All five CVE .c files compile and run through ub-detector without crashing
- ☐ gcc_bug_30475.c and loop_wrap.c both flagged as SignedOverflow
- ☐ cve_2009_1897.c flagged as NullDeref
- ☐ openssl_alias.c outcome documented (caught or explained as out-of-scope)
- ☐ dead_code.c outcome documented with reason
- ☐ Written evaluation note for each case: caught / partial / missed + reason

A Appendix — Quick Reference & Failure Modes

A.1 Complete Bug Fix Summary

#	Bug	File	Fix
1	Shared LLVMContext	main.cpp	Declare ctx0 and ctx2 separately; pass each to compileAt()
2	users() not program order	UBClassifier.cpp	Walk AI->getParent() basic block instructions sequentially
3	nsw alone = false positive	UBClassifier.cpp	Require nsw flag AND (branches_O2 < branches_O0) together
4	touch not on Windows	Phase 0 script	Replace all touch calls with New-Item -ItemType File -Force
5	Silent inlined function skip	BehaviorDiff.cpp	Print [INFO] message when function missing from M_O2
6	Mangled names in report	Reporter.cpp	Call llvm::demangle(r.diff.name) before printing function name

A.2 LLVM API Cheatsheet

Class	Usage in This Project
llvm::LLVMContext	Owns all types. Use ONE per module -- never share between O0 and O2.
llvm::Module	Top-level container. Iterate functions with range-for.
llvm::Function	Iterate basic blocks: for (auto& BB : F)
llvm::BasicBlock	Iterate instructions: for (auto& I : BB) -- this IS program order.
llvm::BranchInst	dyn_cast(&I;) -- .isConditional() for conditional branches.
llvm::BinaryOperator	dyn_cast(&I;) -- .hasNoSignedWrap() for nsw flag.
llvm::ICmpInst	dyn_cast(&I;) -- .getPredicate() for eq/ne/lt/gt.
llvm::AllocaInst	Stack allocation. Walk parent BB in order to find load-before-store.
llvm::DILocation	I.getDebugLoc() -- .getLine(), .getColumn(), .getFilename().
llvm::demangle()	#include -- human-readable C++ names.

A.3 Common Failure Modes

Problem	Fix

compileAt() returns nullptr	Check: (1) filepath is correct, (2) LLVM targets initialized, (3) no source syntax errors.
Random crashes / wrong block counts	Almost certainly Bug #1 (shared context). Verify ctx0 and ctx2 are declared separately.
Too many false positives on safe code	Bug #3 not applied. Ensure hasSignedWrapAdded() checks both nsw AND branch count drop.
uninit.c not caught	Bug #2 not applied. Ensure hasLoadBeforeStore() walks BB sequentially, not users().
DILocation always nullptr	Ensure -g flag is in compileAt() args. Without debug info, locations are unavailable.
nsw flag not appearing	Try: opt -passes='instcombine' -S file_O0.ll and inspect output for nsw.
Function not found in M_O2	Inlining -- now logged by fix #5. Use -fno-inline during development to suppress.

A.4 Build Flags Reference

Flag	Purpose
-fno-inline	Disables inlining. Use during development to prevent functions vanishing at O2.
-g	Emit DWARF debug info. Required for source-level line reporting.
-S -emit-llvm	Emit LLVM IR text (.ll) instead of object file.
-O0	No optimization. Near-literal translation of source code.
-O2	Full optimization. Enables UB-exploiting transforms (nsw, dead block removal).
-fsanitize=undefined	UBSan runtime detection. Complement (not replacement) for this static tool.
-fno-strict-aliasing	Disables aliasing UB assumption. Fix for Pattern 3 (aliasing).